



BODY CONTROL ECU FOR POWER WINDOW AND LIFTGATE APPLICATIONS

In Partial Fulfilment of the Summer Internship Program Requirements

Submitted By

Yash Chaudhari

Sumit Chaudhari

Sahil Gade

Under The Guidance Of

Rahul Sir

Submitted To

KPIT APEX LAB, KPIT Technologies Ltd.

COEP Technological University

May – July 2026

ABSTRACT

The “Body Control ECU for Power Window and Liftgate Applications” project focuses on the design and implementation of an automotive-style Body Control Module (BCM) prototype capable of managing two core vehicle body functions — power window actuation and liftgate (trunk) actuation — using an embedded, state-machine-driven control architecture. The project was undertaken as part of the KPIT Technologies summer internship program and simulates real-world automotive ECU development practices used in the body electronics domain.

The system is built around the STM32 NUCLEO-F446RE development board as the central controller, interfaced with an L293D dual H-bridge motor driver IC for bidirectional control of two 12V Johnson DC motors representing the window and liftgate actuators. Manual control is provided through push-button switches, while an IR sensor enables hands-free, gesture-based operation of the liftgate and doubles as an obstruction/anti-pinch safety sensor. Real-time system status, active operation, and fault or warning messages are displayed on an ST7920 12864 graphic LCD interfaced over SPI.

The firmware was developed in Embedded C using STM32CubeIDE and STM32CubeMX, with HAL drivers used for GPIO, PWM/timer, and SPI peripheral configuration. The control logic follows a state-machine-based design (INIT, IDLE, MOVING/OPENING/CLOSING, FAULT) so that no actuator can be driven without passing through state validation, and all commands — whether from push buttons or the IR gesture sensor — are validated before execution.

The project was carried out over a six-week internship schedule covering system architecture definition, STM32 bring-up, power window control logic, liftgate control logic, LCD and IR sensor integration, fault/safety handling, and final integration testing. The completed prototype successfully demonstrates manual push-button control, IR gesture-based hands-free liftgate operation, obstruction detection with automatic motor stop, and real-time status feedback on the graphic LCD.

The original case study specification additionally called for a touchscreen HMI and CAN-based remote command/diagnostics interface with a PC as a supervisory node; these were treated as extended/future scope and are discussed in the Future Improvements chapter, alongside anti-pinch enhancement and EEPROM-based fault logging.

TABLE OF CONTENTS

ABSTRACT.....	2
TABLE OF CONTENTS	3
CHAPTER 1: INTRODUCTION.....	5
1.1 Objectives	5
1.2 Problem Statement.....	5
CHAPTER 2: LITERATURE REVIEW AND REFERENCE MATERIAL USED.....	6
CHAPTER 3: HARDWARE AND SOFTWARE.....	7
3.1 Introduction	7
3.2 Hardware Used	7
3.2.1 STM32 NUCLEO-F446RE	7
3.2.2 L293D Motor Driver IC	7
3.2.3 Johnson DC Motor (12V, 200 RPM)	7
3.2.4 IR Sensor	7
3.2.5 ST7920 12864 Graphic LCD.....	7
3.2.6 Push Button Switches	7
3.2.7 12V Power Supply	7
3.3 Software Technologies Used	8
3.3.1 STM32CubeIDE	8
3.3.2 STM32CubeMX	8
3.3.3 Embedded C.....	8
3.3.4 HAL Drivers	8
CHAPTER 4: SYSTEM ARCHITECTURE AND PIN CONNECTIONS	9
4.1 System Block Diagram	9
4.2 GPIO Pin Mapping.....	9
4.3 L293D Motor Control Logic.....	9
4.4 Control Architecture	10
CHAPTER 5: HARDWARE SETUP AND SYSTEM PREPARATION (WEEK 1).....	11
5.1 Week 1 Plan	11
5.1 Week 1 Work Progress	11
Work Done	11
Problems Faced.....	11
Solutions Implemented.....	11
CHAPTER 6: POWER WINDOW CONTROL DEVELOPMENT (WEEK 2)	12
6.1 Week 2 Plan	12
6.1 Week 2 Work Progress	12
Work Done	12

Problems Faced.....	12
Solutions Implemented.....	12
CHAPTER 7: LIFTGATE CONTROL DEVELOPMENT (WEEK 3)	13
7.1 Week 3 Plan	13
7.1 Week 3 Work Progress	13
Work Done	13
Problems Faced.....	13
Solutions Implemented.....	13
CHAPTER 8: IR SENSOR AND GESTURE-BASED CONTROL (WEEK 4).....	14
8.1 Week 4 Plan	14
8.1 Week 4 Work Progress	14
Work Done	14
Problems Faced.....	14
Solutions Implemented.....	14
CHAPTER 9: SAFETY, FAULT HANDLING AND STATUS DISPLAY (WEEK 5)	15
9.1 Week 5 Plan	15
9.1 Week 5 Work Progress	15
Work Done	15
Problems Faced.....	15
Solutions Implemented.....	15
CHAPTER 10: INTEGRATION, TESTING AND FINAL DEMONSTRATION (WEEK 6)	16
10.1 Week 6 Plan	16
10.1 Week 6 Work Progress	16
Work Done	16
Problems Faced.....	16
Solutions Implemented.....	16
CHAPTER 11: SAFETY FEATURES	17
Obstacle Detection	17
Motor Stop.....	17
Anti-Pinch Concept	17
Command Validation	17
CHAPTER 12: TESTING AND RESULTS	18
FUTURE IMPROVEMENTS	19
CONCLUSION.....	20
ACKNOWLEDGEMENT.....	21
REFERENCES.....	21

CHAPTER 1: INTRODUCTION

In modern automotive systems, Body Control Modules (BCMs) are responsible for controlling comfort, convenience, and safety-related body functions such as power windows, liftgates, door locks, lighting, and wipers. These systems are implemented as Electronic Control Units (ECUs) that interact with actuators, human-machine interfaces (HMIs), and in some architectures, in-vehicle communication networks such as CAN (Controller Area Network).

This project focuses on the design and implementation of a Body Control ECU that manages two key body functions: Power Window Control and Liftgate (Trunk) Control. The ECU was built to provide motorized bidirectional actuation, manual push-button and gesture-based user interaction, obstruction-based safety cut-off, and real-time status feedback on a graphic LCD — giving hands-on experience with embedded system design, motor control, state-machine-based logic, sensor interfacing, and HMI development following an industry-relevant workflow.

1.1 Objectives

- Design and implement an automotive-style Body Control ECU prototype using the STM32 NUCLEO-F446RE.
- Implement complete, safely-validated power window UP/DOWN/STOP control logic.
- Implement liftgate OPEN/CLOSE/STOP control logic, independent of the power window logic.
- Integrate IR-based gesture detection for hands-free liftgate operation and obstruction sensing.
- Display real-time system status, active operation, and fault/warning messages on a graphic LCD.
- Demonstrate bidirectional motor control using the L293D dual H-bridge driver.
- Apply fault handling and safe-state concepts so that no actuator can be driven without state validation.

1.2 Problem Statement

Traditional power window and liftgate mechanisms rely on simple manual switches with no intelligence, leaving them vulnerable to pinch hazards, accidental operation, and a lack of centralized status feedback. There is a clear need for a smarter, ECU-driven control system that simplifies multi-mode operation, prevents unsafe motor operation through validated states, detects obstruction and reacts automatically, and supports gesture/sensor-driven actuation in addition to manual switches.

CHAPTER 2: LITERATURE REVIEW AND REFERENCE MATERIAL USED

Before development began, the official documentation for each hardware component and development tool was studied to understand correct interfacing, electrical limits, and configuration procedures. Key references included the STM32 NUCLEO-F446RE user manual and datasheet from STMicroelectronics, the STM32CubeIDE and STM32CubeMX user guides, the L293D dual H-bridge motor driver datasheet, and the ST7920 graphic LCD controller datasheet. KPIT's internal case study brief and learning resources for “Body Control ECU for Power Window and Liftgate Applications” were used as the primary project specification and review framework.

This research established the baseline understanding needed for GPIO/PWM/timer configuration on the STM32, correct IN/EN signal sequencing for the L293D H-bridge, SPI-based command sequencing for the ST7920 controller, and general best practices for state-machine-driven embedded control design used in real automotive body control modules.

CHAPTER 3: HARDWARE AND SOFTWARE

3.1 Introduction

This chapter lists the hardware components and software tools used to build the Body Control ECU prototype.

3.2 Hardware Used

3.2.1 STM32 NUCLEO-F446RE

Acts as the central processing and control unit of the ECU. It reads push-button and IR sensor inputs, runs the state-machine control logic, generates PWM signals for motor speed control, and drives the SPI interface to the graphic LCD.

3.2.2 L293D Motor Driver IC

A dual H-bridge motor driver IC used to provide bidirectional drive for both the power window motor and the liftgate motor from two independent channels. IN1/IN2 (window) and IN3/IN4 (liftgate) set rotation direction, while the EN1/EN2 PWM inputs control motor speed.

3.2.3 Johnson DC Motor (12V, 200 RPM)

Two 12V, 200 RPM Johnson DC motors are used to simulate the power window and liftgate actuators respectively.

3.2.4 IR Sensor

Used for two purposes: hand-wave gesture detection for hands-free liftgate opening, and obstacle/obstruction detection during window and liftgate motion (anti-pinch concept). The sensor output is processed via a STM32 GPIO interrupt/ADC input and triggers an immediate motor stop when an obstruction is detected.

3.2.5 ST7920 12864 Graphic LCD

A graphic LCD interfaced over SPI that displays live system status — active operation (window/liftgate opening or closing), obstacle/fault warnings, motor-stop confirmation, and command source — giving the user feedback without needing to observe the actuator directly.

3.2.6 Push Button Switches

Provide manual override control input for window UP/DOWN and liftgate OPEN/CLOSE operations, continuously polled by the STM32 GPIO.

3.2.7 12V Power Supply

Supplies power to the DC motors and supporting logic circuitry.

3.3 Software Technologies Used

3.3.1 STM32CubeIDE

The primary Integrated Development Environment used for firmware development, build, flashing, and debugging on the STM32 NUCLEO-F446RE.

3.3.2 STM32CubeMX

Used for peripheral and clock configuration — GPIO modes, PWM timer channels, SPI interface, and interrupt configuration — with auto-generated HAL initialization code.

3.3.3 Embedded C

The core programming language used to implement the state-machine control logic, motor control routines, sensor processing, and LCD display routines.

3.3.4 HAL Drivers

STM32 Hardware Abstraction Layer drivers were used to interface with GPIO, timer/PWM, and SPI peripherals in a portable, modular manner.

CHAPTER 4: SYSTEM ARCHITECTURE AND PIN CONNECTIONS

4.1 System Block Diagram

The STM32 NUCLEO-F446RE sits at the centre of the system architecture. It interfaces with the L293D motor driver (which in turn drives the power window motor and liftgate motor), an IR sensor for gesture and obstacle detection, push-button switches for manual override, and the ST7920 12864 graphic LCD for real-time status display. Power/motor signal paths and digital I/O/sensor paths are kept electrically and logically separate for safety.

4.2 GPIO Pin Mapping

The following table lists the complete pin assignment used on the STM32 NUCLEO-F446RE for this project:

Pin	Function	Description
PA0	Motor IN1	Power window motor – H-bridge direction input 1
PA1	Motor IN2	Power window motor – H-bridge direction input 2
PA2	Liftgate IN3	Liftgate motor – H-bridge direction input 3
PA3	Liftgate IN4	Liftgate motor – H-bridge direction input 4
PA4	IR Sensor	Gesture / obstacle detection input
PA8	EN1 (PWM)	Power window motor – speed control (PWM enable)
PA9	EN2 (PWM)	Liftgate motor – speed control (PWM enable)
PB12	LCD CS	ST7920 LCD – chip select (SPI)
PB13	LCD CLK	ST7920 LCD – serial clock (SPI)
PB14	LCD RST	ST7920 LCD – reset
PB15	LCD SID	ST7920 LCD – serial data in (SPI)
PC0	Window Switch	Push-button input – power window UP/DOWN
PC1	Liftgate Switch	Push-button input – liftgate OPEN/CLOSE

4.3 L293D Motor Control Logic

The L293D provides two independent H-bridge channels for bidirectional drive. IN1/IN2 (window channel) and IN3/IN4 (liftgate channel) set the rotation direction, while the corresponding EN pin's PWM signal controls motor speed. Reversing the IN states reverses motor rotation direction.

IN1	IN2	Motor Action
HIGH	LOW	Rotate clockwise (Window UP)
LOW	HIGH	Rotate anticlockwise (Window DOWN)
LOW	LOW	Stop (free-run)

The same truth table logic applies to IN3/IN4 for the liftgate channel (OPEN corresponds to clockwise rotation, CLOSE to anticlockwise rotation).

4.4 Control Architecture

The firmware follows a state-machine-based design so that direct actuator control without state validation is never permitted. The typical states used are:

- INIT – system power-up and peripheral initialization
- IDLE – waiting for a valid push-button or IR gesture command
- MOVING / OPENING / CLOSING – motor actively driven in the commanded direction
- FAULT – entered on obstruction detection, timeout, or invalid command; forces immediate motor stop

CHAPTER 5: HARDWARE SETUP AND SYSTEM PREPARATION (WEEK 1)

5.1 Week 1 Plan

- System understanding and architecture definition
- STM32 NUCLEO-F446RE bring-up
- GPIO, PWM, and timer configuration in STM32CubeMX
- Basic motor rotation test using the L293D driver

5.1 Week 1 Work Progress

Work Done

- Studied the KPIT case study brief and defined the overall Body Control ECU architecture, including the choice of STM32 NUCLEO-F446RE, L293D motor driver, IR sensor, push buttons, and ST7920 LCD.
- Brought up the STM32 NUCLEO-F446RE board in STM32CubeIDE and verified basic project build, flash, and debug workflow via ST-LINK.
- Configured GPIO output pins (PA0, PA1, PA2, PA3) for L293D IN1–IN4 and PWM timer channels (PA8, PA9) for EN1/EN2 speed control in STM32CubeMX.
- Wired the L293D to the STM32 and one DC motor, and successfully performed a basic clockwise/anticlockwise motor rotation test to validate the H-bridge wiring and GPIO logic.

Problems Faced

1. Pin conflicts were encountered between peripherals while allocating GPIO pins in STM32CubeMX for the motor driver, sensor, and LCD interfaces.
2. Intermittent ST-LINK debugger detection issues occurred during flashing, occasionally requiring the board to be reconnected.

Solutions Implemented

3. Re-mapped conflicting pins by reviewing the NUCLEO-F446RE pinout diagram and re-generating the CubeMX project with a conflict-free pin allocation (see Section 4.2).
4. Resolved ST-LINK detection issues by updating the ST-LINK firmware and using a shorter, verified USB cable for flashing.

CHAPTER 6: POWER WINDOW CONTROL DEVELOPMENT (WEEK 2)

6.1 Week 2 Plan

- Power window control logic implementation
- Basic state-machine design
- ST7920 LCD bring-up and basic status display

6.1 Week 2 Work Progress

Work Done

- Implemented push-button polling on PC0 for power window UP/DOWN commands, with PWM-based motor speed control via EN1.
- Designed and implemented the initial state-machine (INIT → IDLE → MOVING) so that button presses are validated against the current state before the motor is driven.
- Added automatic motor stop logic on command timeout and simulated end-of-travel condition.
- Brought up the ST7920 12864 graphic LCD over SPI (CS/CLK/RST/SID on PB12–PB15) and displayed basic static status text.

Problems Faced

5. Getting the ST7920 SPI timing and initialization sequence correct was difficult — the LCD initially showed garbled or blank output.
6. Motor speed response felt inconsistent when the PWM duty cycle was changed.

Solutions Implemented

7. Corrected the ST7920 initialization sequence and SPI clock timing by cross-checking against the controller datasheet, which resolved the display issue.
8. Tuned the PWM frequency and duty cycle range to match the L293D EN input characteristics, producing smooth and consistent motor speed control.

CHAPTER 7: LIFTGATE CONTROL DEVELOPMENT (WEEK 3)

7.1 Week 3 Plan

- Liftgate control logic implementation
- Multi-state handling (OPEN/CLOSE/STOP)
- LCD status screen enhancements

7.1 Week 3 Work Progress

Work Done

- Implemented liftgate OPEN/CLOSE/STOP control on IN3/IN4/EN2, using push-button input on PC1 as the manual control source.
- Extended the state machine to independently handle liftgate OPENING/CLOSING states in parallel with the power window states, with mutual exclusion logic to prevent conflicting simultaneous commands.
- Enhanced the LCD to show which subsystem (window or liftgate) is currently active and its current operation.

Problems Faced

9. Occasional motor driver integration issues were faced in matching the PWM frequency and EN logic between the window and liftgate channels of the L293D.
10. Simultaneous button presses for window and liftgate occasionally caused ambiguous motor behaviour.

Solutions Implemented

11. Standardized a single PWM frequency and EN enable/disable sequence shared across both L293D channels, eliminating the inconsistent behaviour.
12. Added mutual-exclusion checks in the state machine so that a liftgate command is ignored while the window subsystem is active, and vice versa.

CHAPTER 8: IR SENSOR AND GESTURE-BASED CONTROL (WEEK 4)

8.1 Week 4 Plan

- IR sensor integration for gesture and obstacle detection
- Hands-free liftgate gesture control
- Obstruction-triggered motor stop logic

8.1 Week 4 Work Progress

Work Done

- Integrated the IR sensor on PA4 for hand-wave gesture detection, enabling hands-free liftgate opening as an alternative to the push button.
- Implemented obstacle detection logic so the same IR sensor is monitored continuously during window and liftgate motion.
- Implemented an anti-pinch concept: on detecting an obstruction mid-travel, the motor is stopped immediately and, where applicable, briefly reversed.

Problems Faced

13. Tuning the IR sensor threshold for reliable gesture detection without false triggers from ambient light or unrelated motion was challenging.
14. Obstruction detection sometimes triggered false stops due to sensor noise.

Solutions Implemented

15. Calibrated the IR sensor threshold empirically across multiple test distances and lighting conditions to achieve reliable gesture recognition with minimal false triggers.
16. Added basic debounce/averaging on the IR sensor reading before triggering a FAULT/obstruction state, eliminating spurious stops.

CHAPTER 9: SAFETY, FAULT HANDLING AND STATUS DISPLAY (WEEK 5)

9.1 Week 5 Plan

- Fault handling implementation across window and liftgate subsystems
- Timeout-based safety logic
- Full LCD status/diagnostic screens

9.1 Week 5 Work Progress

Work Done

- Implemented a dedicated FAULT state entered on obstruction detection, command timeout, or invalid state transition, forcing an immediate, validated motor stop.
- Added timeout-based safety logic so that a motor cannot be driven continuously beyond a safe duration without a fresh valid command.
- Completed the LCD status screens: WINDOW OPENING, WINDOW CLOSING, LIFTGATE OPENING, OBSTACLE DETECTED, and MOTOR STOPPED, along with a command-source indicator (button/gesture).

Problems Faced

17. Coordinating FAULT-state recovery between the window and liftgate subsystems without one blocking the other required careful state design.
18. LCD screen updates occasionally lagged behind the actual motor state during rapid button presses.

Solutions Implemented

19. Refactored the state machine so FAULT handling is subsystem-scoped (window and liftgate each maintain independent fault flags) rather than global, allowing one subsystem to recover independently of the other.
20. Optimized the LCD update routine to run on a fixed refresh cycle synchronized with the state machine tick, removing the display lag.

CHAPTER 10: INTEGRATION, TESTING AND FINAL DEMONSTRATION (WEEK 6)

10.1 Week 6 Plan

- Full system integration testing
- Bug fixing and optimization
- Documentation and final demo preparation

10.1 Week 6 Work Progress

Work Done

- Performed end-to-end integration testing of the power window, liftgate, IR gesture, obstruction detection, and LCD status subsystems together on the assembled prototype.
- Verified that all commands — push-button and IR gesture — are correctly validated against the current state before any actuator is driven.
- Prepared the final project documentation, GPIO pin mapping tables, and recorded demonstration evidence of window opening, liftgate opening, LCD status display, and IR gesture/obstruction detection in action.

Problems Faced

21. A few loose wiring connections on the breadboard surfaced intermittently during extended integration testing.
22. Minor timing mismatches appeared between the IR obstruction check and the PWM motor-stop response during stress testing.

Solutions Implemented

23. Identified and re-secured all loose connections after a full hardware continuity check, and moved critical connections to a more secure prototyping setup.
24. Reduced the IR sensor polling interval and tightened the motor-stop response path in the control loop, eliminating the timing mismatch.

CHAPTER 11: SAFETY FEATURES

The following safety mechanisms were designed into the Body Control ECU:

Obstacle Detection

The IR sensor continuously monitors the path of motion of the window and liftgate during operation.

Motor Stop

Immediate motor cut-off is triggered on fault detection or obstruction, driven through the state machine's FAULT state.

Anti-Pinch Concept

On sensing resistance/obstruction, the affected motor is stopped and briefly reversed to relieve any pinch condition.

Command Validation

All commands, whether from push buttons or the IR gesture sensor, are validated against the current system state before being executed — direct, unvalidated actuator control is never permitted.

CHAPTER 12: TESTING AND RESULTS

The completed prototype was tested and demonstrated for the following scenarios, with photo/video evidence captured during the final demonstration:

- Power window UP/DOWN operation via push button, including automatic stop at simulated end-of-travel.
- Liftgate OPEN/CLOSE operation via push button and via IR hand-wave gesture.
- Obstacle detection during window and liftgate motion, with immediate motor stop and LCD warning display.
- Real-time LCD status updates for all active operations, faults, and command sources.
- Mutual exclusion verified — simultaneous conflicting window/liftgate commands were correctly rejected by the state machine.

FUTURE IMPROVEMENTS

The original KPIT case study specification additionally called for a touchscreen-based HMI (with dedicated screens for power window, liftgate, and system diagnostics) and CAN-based communication, with a PC running PCAN-USB acting as a supervisory/diagnostic node capable of sending window and liftgate commands and receiving ECU status, actuator state, fault codes, and periodic heartbeat messages. These were treated as extended/future scope for this build and are natural next steps:

- Add a CAN transceiver and implement the CAN message dictionary (RX: window/liftgate/reset commands; TX: ECU status, actuator state, fault codes, periodic heartbeat) with a PC/PCAN-USB supervisory node.
- Replace/augment the push-button and LCD status interface with a touchscreen HMI offering dedicated control screens for power window, liftgate, and system diagnostics.
- Extend the anti-pinch logic with force/current-sensing feedback in addition to IR-based obstruction detection, for more sensitive pinch protection.
- Add EEPROM-based fault and event logging so that fault history persists across power cycles.
- Scale the prototype to additional body functions such as door locks and exterior lighting on the same hardware/software architecture.

CONCLUSION

The “Body Control ECU for Power Window and Liftgate Applications” project successfully demonstrates a working Body Control ECU prototype that integrates bidirectional motor control, push-button and IR gesture-based user interaction, obstruction-based safety cut-off, and real-time status display on a single STM32 NUCLEO-F446RE platform.

The project addressed the need for an intelligent, ECU-driven alternative to traditional manual power window and liftgate switches, and provided practical, hands-on experience with embedded C programming, state-machine design, GPIO/PWM/SPI peripheral interfacing, motor driver integration, sensor calibration, and automotive body electronics concepts.

Despite challenges such as GPIO pin conflicts, ST-LINK detection issues, LCD SPI timing debugging, motor driver integration, and IR sensor calibration, the project successfully achieved a functional, safely state-validated Body Control ECU. With the addition of CAN communication and a touchscreen HMI as outlined in the Future Improvements chapter, the system has clear potential to evolve toward the full scope of the original KPIT case study specification.

ACKNOWLEDGEMENT

The successful completion of this internship project would not have been possible without the guidance, support, and encouragement received from various individuals and organizations throughout the development process.

Sincere gratitude is expressed to KPIT Technologies Ltd. and KPIT APEX Lab for providing the opportunity to work on the project titled “Body Control ECU for Power Window and Liftgate Applications” as part of the internship program, and for valuable exposure to real-world automotive body electronics and ECU development practices.

Special thanks are extended to our KPIT mentor for providing the project problem statement, continuous technical guidance, and valuable suggestions throughout the internship duration.

Gratitude is also expressed to COEP Technological University and to faculty members, coordinators, and colleagues who provided support during hardware setup, software debugging, testing, and documentation activities throughout the internship period.

Special appreciation is given to the developers and contributors of STM32CubeIDE, STM32CubeMX, and the HAL driver libraries, which made implementation of this project possible on affordable embedded hardware.

Finally, heartfelt thanks are expressed to family members for their continuous encouragement and support during the successful completion of this project and its documentation.

REFERENCES

1. STMicroelectronics, "UM1724 – STM32 Nucleo-64 Boards (MB1136) User Manual," 2025.
2. STMicroelectronics, "STM32 NUCLEO-F446RE – Official Datasheet and Documentation."
3. STMicroelectronics, "STM32CubeIDE / STM32CubeMX User Guides."
4. Texas Instruments, "L293D Dual H-Bridge Motor Driver – Datasheet."
5. "ST7920 Graphic LCD Controller – Datasheet."
6. KPIT Technologies Ltd., "Case Study Details & Guidelines – Body Control ECU for Power Window and Liftgate Applications," Version 1.0.
7. KPIT Technologies – Internal Case Study & Learning Resources.